Protocol Evaluation on Teleoperation

Abstract

This project investigates the impact of transport layer protocol choices (TCP, UDP, WebRTC) on the Quality of Experience (QoE) in robotic teleoperation. On perfect localized networks, these protocols act similarly, but their differences emerge decisively under network degradation and packet loss. We evaluated command flows (non-idempotent delta positions) and state flows (idempotent absolute positions) using pure TCP, pure UDP, and parameterized WebRTC DataChannels. Our experiments simulated various 4G network conditions, including extreme intervals of 100% packet loss. We demonstrate that relying entirely on a single protocol yields either massive network latency spikes (TCP Head-of-Line blocking) or fatal robotic desynchronization/drift (UDP packet drops). The findings prove that optimal QoE is achieved through a hybrid approach: reliable, ordered transmission for command flows and unreliable, unordered transmission for state flows.

1 Introduction / Project Recap

Problem Statement: In remote robotic teleoperation, unpredictable network conditions like packet loss critically degrade the operator's Quality of Experience (QoE).
Motivation: Choosing the wrong transport protocol can lead to either an unresponsive visual feed or permanent physical desynchronization (drift) between the digital twin and the real robot.
Objectives:

1. Evaluate Pure TCP, Pure UDP, and WebRTC hybrid flows under degraded networks.
2. Quantify QoE metrics including Stutter/Smoothness (State Jitter), Positional Drift, End-to-End Delay, and Stalls.
3. Validate the necessity of matching protocol reliability to data idempotency.

Project Type: Systems Evaluation and Networking Analysis.

2 System Model/Architecture

The system consists of a digital twin interface and a simulated robotic environment operating over an emulated variable network.

- **Components:** Unity-based Client (operator interface), Isaac Sim Server (UR3 robotic arm execution), and an intermediate network emulation layer (`tc_replay.py`). WebRTC experiments further utilize a Python signaling server.

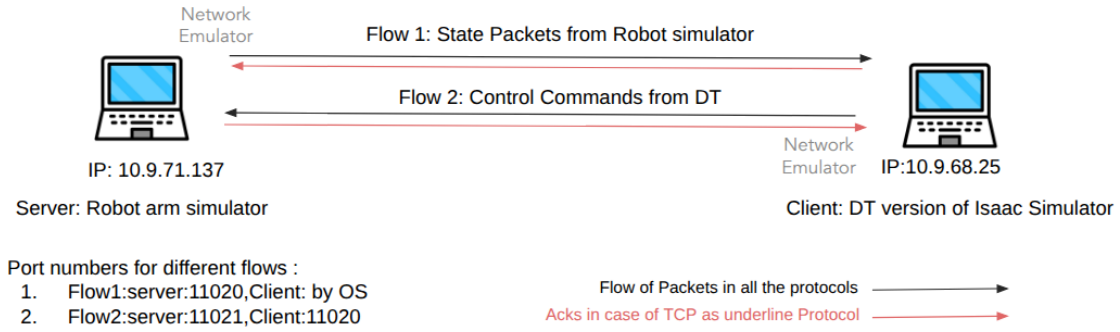

Figure 1: Flow Diagram (Unity Client, Isaac Server the split between Command/State flows

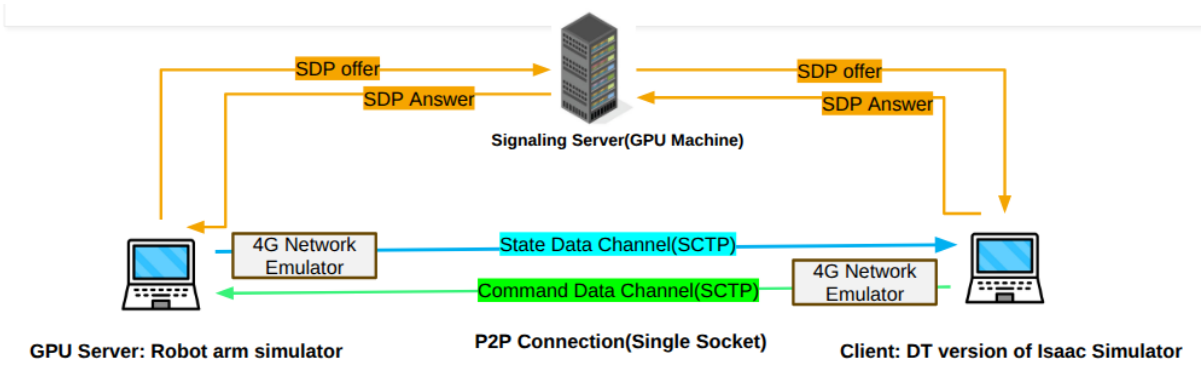

Figure 2: Flow Diagram (Unity Client, Isaac Server, WebRTC Signaling Server, and the split between Command/State flows)

- **Assumptions/Constraints:** State data represents absolute joint positions (idempotent and transient), while Command data represents relative positional deltas (non-idempotent, requiring strict order).
- **Threat Model:** The primary environmental "threats" simulated are severe packet loss and network bandwidth starvation typical of unstable 4G uplinks.

3 System Architecture and Methodology

The system separates the bilateral teleoperation flow into distinct uplinks and downlinks. Commands are transmitted from the Client to the Server, while State payloads return from the Server to the Client. Under WebRTC configurations, DataChannels run over UDP utilizing SCTP.

The methodology tests pure protocol modes and mixed-protocol simulations via WebRTC DataChannel parameters (toggling `ordered` and `maxRetransmits`).

4 Design Logic

The core mechanism requires the client to dispatch continuous delta commands (e.g., +0.05 radians to the base joint) while concurrently ingesting absolute position updates from the server. To systematically validate the impact of transport reliability versus latency, the system is designed around distinct configuration sets separating strict OS-level protocols from WebRTC DataChannel behaviors.

4.1 Pure Protocol Validation

These configurations test raw socket behaviors by mixing transport protocols for the command (uplink) and state (downlink) streams:

- **Command TCP, State TCP:** Both channels enforce strictly guaranteed, ordered delivery with OS-level retransmissions.
- **Command TCP, State UDP:** Commands enforce strict reliability and ordering; State data uses raw, fire-and-forget datagrams.
- **Command UDP, State TCP:** Commands use raw, fire-and-forget datagrams; State data enforces strict reliability and ordering.
- **Command UDP, State UDP:** Both channels operate with raw, fire-and-forget datagrams lacking any retransmission logic.

4.2 WebRTC Validation

These configurations utilize WebRTC SCTP-over-UDP DataChannels. Protocol behaviors are simulated by toggling `ordered` and `maxRetransmits` parameters:

- **Command TCP behaviour, State UDP behaviour:** Commands simulate TCP (`ordered=True`, `maxRetransmits=None`); State data simulates UDP (`ordered=False`, `maxRetransmits=0`).
- **Command TCP behaviour, State TCP behaviour:** Both channels simulate strictly reliable, ordered TCP connections.
- **Command UDP behaviour, State TCP behaviour:** Commands simulate unreliable UDP; State data simulates strictly reliable TCP.
- **Command UDP behaviour, State UDP behaviour:** Both channels simulate unreliable, unordered UDP connections.

5 Implementation Details

- **Client:** Unity application (C#), utilizing custom UDP/TCP sockets and WebRTC endpoints.
- **Server:** Isaac Sim environment controlled via Python scripts (`server_webrtc.py`, `server_CTCP_SUDP.py`, etc.).
- **Network Emulation:** `tc_replay.py` is applied at both client and server uplinks to replay actual 4G bandwidth capacity traces (e.g., `bicycle_001_throughput`).
- **WebRTC Details:** Reliable channels operate with `ordered=True` & `maxRetransmits=None`. Unreliable channels use `ordered=False` & `maxRetransmits=0`.

6 Experimental Setup

- **Action:** The robot is instructed to execute continuous positional delta operations (0.05 rad) strictly to the base joint via an automated script.
- **Network Variants:**
  1. **Baseline:** Unrestricted network.
  2. **Network Variant 1:** 4G emulation using `bicycle_001_throughput`.
  3. **Network Variant 2:** 4G emulation using `bicycle_002_throughput`, characterized by a 10-second window of 0 throughput (effectively 100% packet loss).

7 Results and Analysis

7.1 Network RTT

TCP state transmission experiences significant trailing network delay while recovering lost packets, whereas UDP state data bounds the maximum latency, delivering better overall real-time QoE and fewer visual stalls.

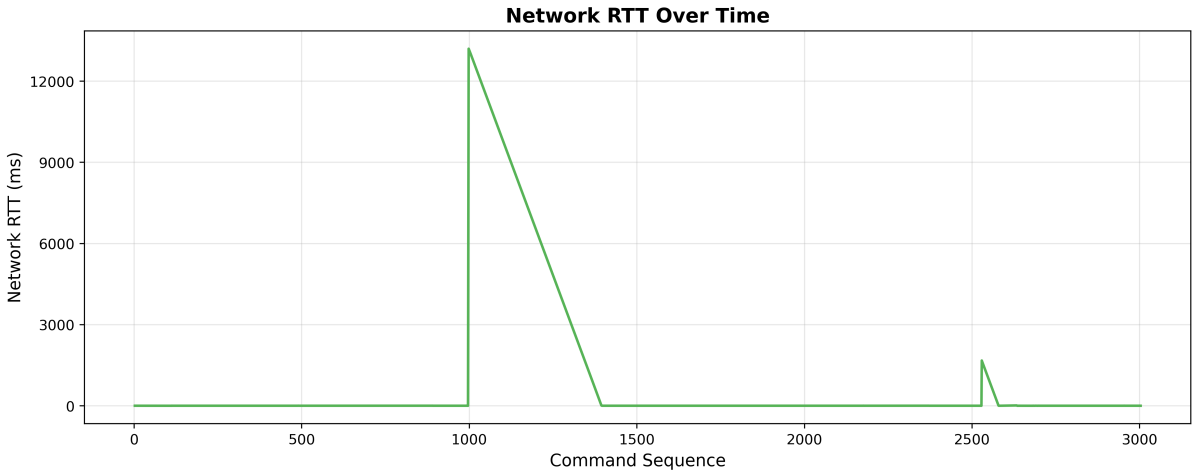

Figure 3: TCP Commands

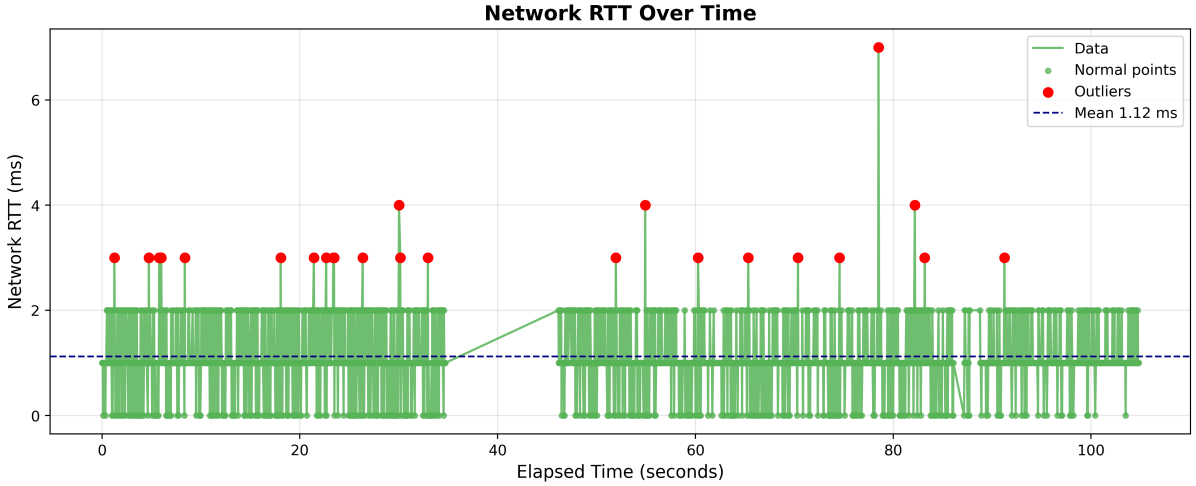

Figure 4: UDP Commands

Network RTT

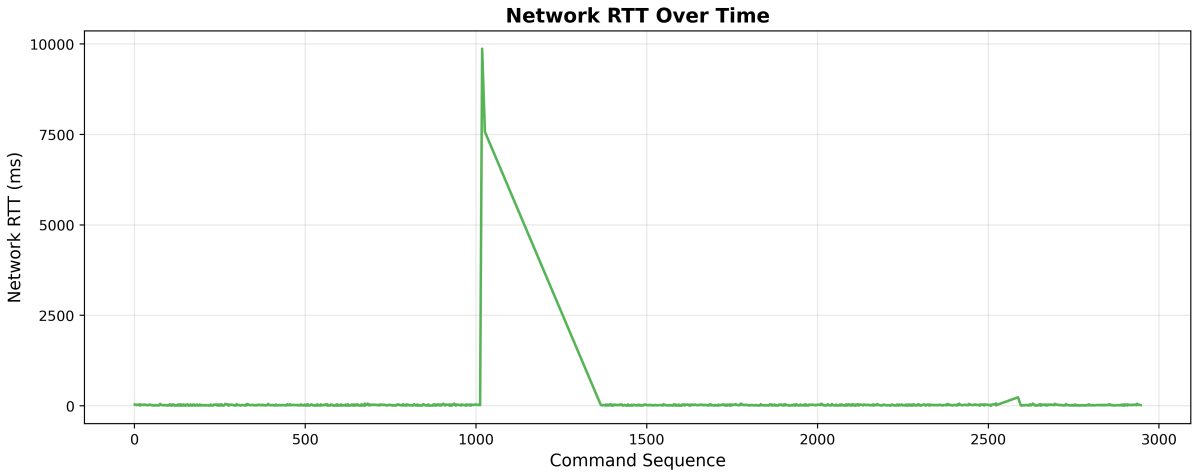

Figure 5: WEBRTC TCP

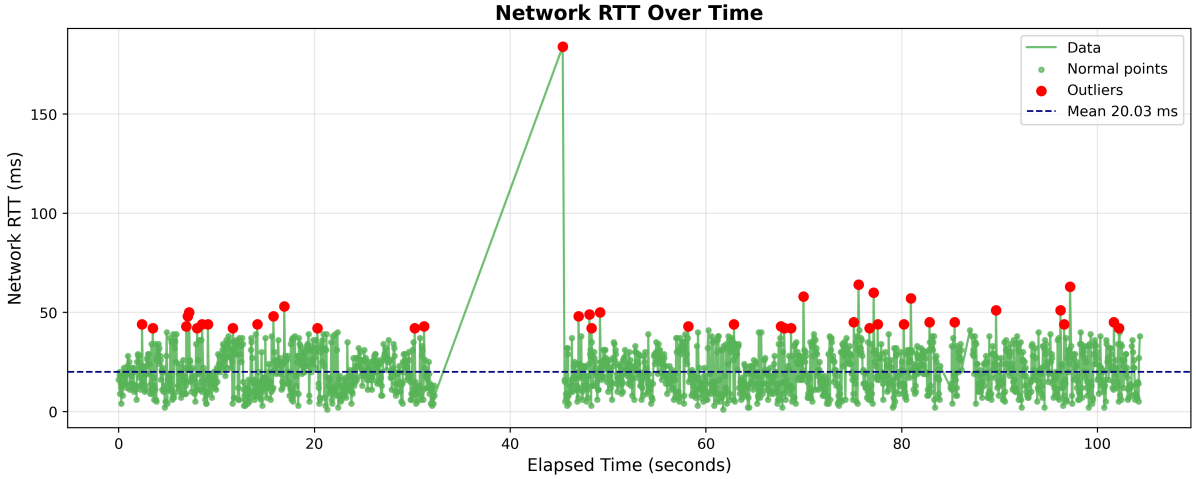

Figure 6: WEBRTC UDP

Network RTT

7.2 Positional Drift (Command Desynchronization)

- **Config B (UDP):** Because commands are unreliable, a 5% packet loss means ∼5 of your delta commands simply vanished. The Unity Robot Actual commands shows +1.0 rad sent, but based on state data received from the Isaac server, the unity robot actually only moved to +0.95 rad. **Bad QoE (Robotic Drift).**

- **Config A & C (TCP/Hybrid):** Because commands are reliable, missing packets are retransmitted. It might take an extra 50ms for the command to arrive, but eventually, Unity actually moved exactly +1.0 rad, completely eliminating drift and matching the actual commands sent. **Good QoE.**

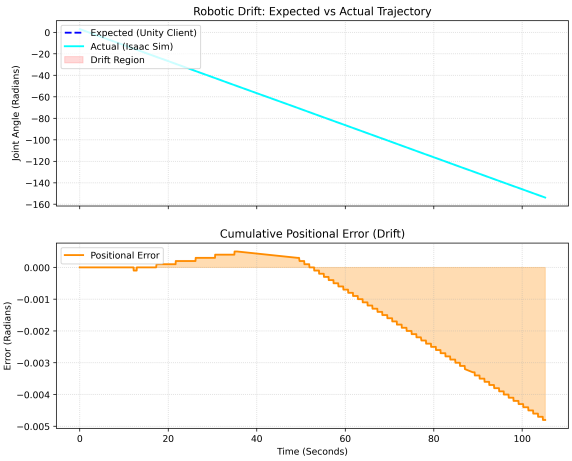

Figure 7: TCP Commands (No Drift)

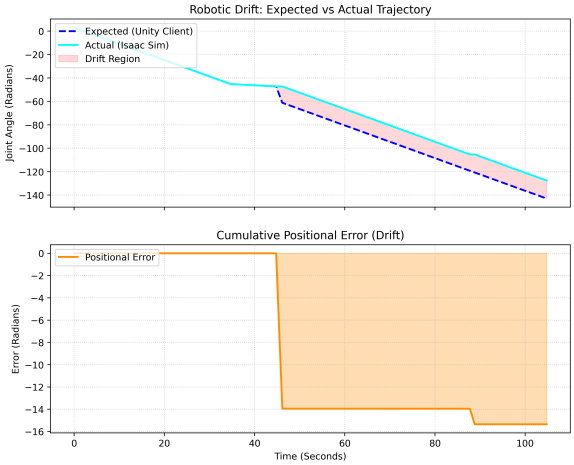

Figure 8: UDP Commands (Severe Drift)

Positional Drift Comparison: Actual vs Expected arm angles for TCP and UDP configurations.

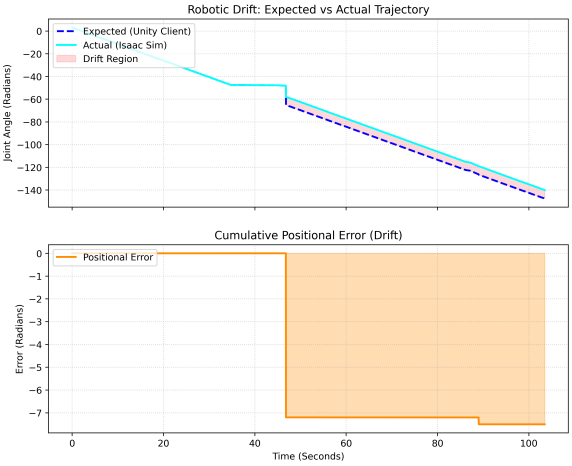

Figure 9: WEBRTC TCP (Minor Drift)

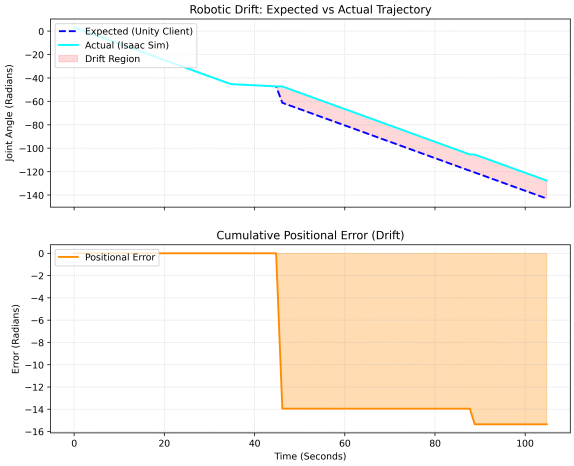

Figure 10: WEBRTC UDP (Severe Drift)

Positional Drift Comparison: Actual vs Expected arm angles for WEBRTC TCP and UDP configurations.

7.3 Latency Analysis

Table 1: Performance Metrics across UDP and TCP for Baseline and Network Variants

Metric	UDP			TCP		
	BaseLine	**Variant 1**	**Variant 2**	**BaseLine**	**Variant 1**	**Variant 2**
Latency						
Control Loop Latency (CLL)	20.77	24.31	25.71	17.835	17.751	23.513
AppBuffer Delay	17.31	18.77	17.35	16.896	16.791	16.791
Rendering	17.76	18.67	17.55	17.239	17.161	16.966
Processing Delay	2.35	4.41	7.22	0.049	0.042	0.052
Network RTT	1.10	1.12	1.14	0.939	0.961	6.777
MTP	38.53	42.97	43.26	35.270	35.100	34.750
Jitter	12.77	14.71	12.35	16.896	16.791	16.791
Packet Loss						
Server to Client	0	0	273.67	0	0	0
Client to Server	0	0	305.00	0	0	0

Table 2: WebRTC Performance Metrics across UDP and TCP Behaviors

Metric	UDP behavior *(ordered=false & maxRetransmits=0)*			TCP behavior *(ordered=true & maxRetransmits=Inf)*		
	BaseLine	**Variant 1**	**Variant 2**	**BaseLine**	**Variant 1**	**Variant 2**
Latency (ms)						
Control Loop Latency (CLL)	39.85	40.10	44.40	40.13	44.05	56.73
AppBuffer Delay	16.77	17.07	17.81	17.08	18.32	17.68
Rendering	33.56	34.09	33.69	33.65	38.10	33.78
Processing Delay	2.09	2.44	2.78	1.84	3.77	4.65
Network RTT	20.99	20.59	23.81	21.21	21.96	34.40
MTP	73.41	74.19	78.09	73.78	82.15	90.51
Jitter	15.99	16.80	23.86	15.97	19.27	23.90

Observations on Network Variant 2 :

- **TCP:** Exhibits significant spikes in network latency (Network RTT) as packets are queued behind dropped segments awaiting retransmission, but maintains absolutely zero positional drift.
- **UDP:** Bounds maximum latency beautifully by abandoning dropped packets, keeping real-time responsiveness high, but suffers from fatal positional drift as command delta logic is completely erased.
- **WebRTC (TCP behavior):** Maintained relatively stable latency properties on state channels compared to pure TCP, though it exhibited anomalous minor drift likely due to underlying SCTP buffer timeouts over UDP.

8 Limitations and Failure Analysis

While the WebRTC Hybrid configuration theoretically solves the UDP drift and TCP jitter paradox, experimental data found that WebRTC TCP-behavior channels still suffered from minor positional drift during the complete 10-second outages. This suggests that

the underlying SCTP-over-UDP implementation possesses hard timeout limits or buffer expirations that fail to mimic perfectly infinite TCP persistence under extreme outages.

9 Evaluation Against Objectives

Objective	Outcome	Evidence
Inject packet loss to stress protocols	Achieved	`tc_replay.py` successfully replicated 4G dropouts.
Determine cause of Stuttering	Achieved	Identified TCP Head-of-Line blocking in state data.
Determine cause of Robotic Drift	Achieved	Identified UDP packet loss in non-idempotent delta data.
Identify Optimal Architecture	Achieved	Protocol choice must map to data idempotency (Hybrid logic).

10 Demo Description

The demo showcases the Unity client side-by-side with the Isaac Sim backend. By triggering "continuous_send", the arm rotates. When the `tc_replay.py` script hits the 100% packet loss window (Network Variant 2), the pure TCP configuration demo visually freezes for several seconds in Unity before abruptly jumping. In contrast, the UDP configuration continues visually tracking, but upon resumption, the physical robot stops short of the intended destination (desynchronization).

11 Conclusions

The nature of robotic teleoperation data dictates protocol suitability based on *Idempotency*. State Flow (Absolute Joint Positions) is highly idempotent and highly sensitive to latency. Dropped state packets are irrelevant since subsequent packets overwrite them; thus, UDP is highly suitable, and TCP is critically flawed due to HoL blocking. Conversely, Command Flow (Deltas) is completely non-idempotent. Erased commands lead to permanent drift; thus, TCP (or perfectly reliable WebRTC) is mandatory. The optimal architecture uses a WebRTC Reliable DataChannel for inputs and an Unreliable DataChannel for state tracking.

12 Future Work

- **Video and Audio Integration:** Try integrating real-time video and audio data into the current setup to observe how different protocols behave under increased network load and varying loss conditions.
- **SCTP Fine-Tuning:** Investigate the buffering limitations of WebRTC SCTP data channels that caused slight drift under the 10-second outage.
- **Dynamic Data Switching:** Implement logic that dynamically transitions between TCP-like and UDP-like delivery based on inferred network state.

13 References

- NVIDIA Isaac Sim Documentation
- Unity Documentation
- Project GitHub Repository